

COMPENDIO C · DESARROLLO DE SOFTWARE Y PROGRAMACION

UML, traductores, historia de los lenguajes, programacion estructurada, paradigmas y web clasica

Alcance de este documento: síntesis de estudio propia elaborada para la preparación del concurso. **NO reproduce ni reemplaza las obras originales** de la bibliografía oficial del Programa Intelectual de Ingreso del CMN (obras protegidas por derecho de autor de sus respectivos editores). Para el texto completo consulte los originales indicados en la bibliografía.

1. Desarrollo del software y UML

Ciclo de vida	Idea central
Cascada	Fases lineales (requisitos, analisis, diseno, codificacion, pruebas, explotacion); rigido, documentacion intensa
Iterativo / Incremental	Versiones sucesivas que agregan funcionalidad; feedback temprano
Espiral (Boehm)	Ciclos de 4 cuadrantes con evaluacion de riesgos en cada vuelta

- **UML** (Unified Modeling Language): lenguaje de modelado estandar (OMG), **no** un metodo ni un lenguaje de programacion.
- **Diagramas estructurales** (estaticos): clases (atributos, operaciones, relaciones de asociacion, agregacion, composicion, herencia), componentes, objetos, despliegue, paquetes.
- **Diagramas de comportamiento** (dinamicos): casos de uso (actores y escenarios), secuencia (interaccion ordenada en el tiempo), actividades (flujos con decisiones y paralelismo), estados (transiciones por eventos).

2. Traductores, compiladores y compaginadores

Herramienta	Funcion
Compilador	Traduce todo el codigo fuente a codigo objeto antes de ejecutar; deteccion de errores en fase de compilacion; binario autonomo (C, Pascal)
Interprete	Traduce y ejecuta instruccion por instruccion en tiempo real; sin binario; mas lento y flexible (BASIC clasico, VBScript, PHP clasico)
Ensamblador	Traduce mnemonicos de assembler a lenguaje maquina 1 a 1
Compaginador / Enlazador (Linker)	Resuelve referencias cruzadas entre modulos compilados y bibliotecas (estaticas: se incrustan; dinamicas DLL: se resuelven en carga) y ensambla el ejecutable

Fases del compilador: **analisis lexico** (tokens) → **sintactico** (arbol AST) → **semantico** (tipos, ambitos) → **optimizacion** → **generacion de codigo objeto**. Tabla de simbolos y manejo de errores atraviesan todo el proceso.

3. Historia de los lenguajes de procesamiento

Aprox.	Lenguaje	Aporte
1940s	Lenguaje maquina / ensamblador	Programacion directa en binario y mnemonicos
1957	FORTRAN (Backus, IBM)	Primer lenguaje de alto nivel compilado; computo cientifico
1958	LISP	Primer lenguaje funcional; listas y recursion
1959	COBOL	Procesamiento de datos comerciales; registros y archivos
1960	ALGOL 60	Base sintactica de casi todos los lenguajes; bloques y ambito
1964	BASIC	Ensenanza e interpretes interactivos
1970	Pascal (Wirth)	Programacion estructurada didactica
1972	C (Ritchie, Bell Labs)	Sistema operativo UNIX; acceso a memoria con punteros
1974	SQL (Chamberlin, IBM)	Lenguaje declarativo de bases de datos
1983	C++ (Stroustrup)	POO sobre C; clases, herencia, plantillas
1991	Python	Legibilidad, multi-paradigma, scripts

Aprox.	Lenguaje	Aporte
1995	Java / JavaScript / PHP	Maquina virtual JVM; Netscape (Eich); web server-side (Lerdorf)
2000	C# (.NET)	Plataforma gestionada de Microsoft

4. Programacion estructurada

- Corriente formalizada por **Dijkstra** ("Go To Statement Considered Harmful", 1968), con base en el **teorema de Bohm-Jacopini**: todo algoritmo puede expresarse con solo **3 estructuras de control**: secuencia, seleccion (decision) e iteracion.
- El objetivo es claridad y verificabilidad: sin saltos arbitrarios (GOTO), diseño **descendente (top-down)**, refinamiento sucesivo y modulos con **alta cohesion** y **bajo acoplamiento**.

5. Paradigmas de programacion (6 del Programa CMN)

Paradigma	Concepto	Ejemplos de lenguajes
Imperativo	Programa como secuencia de ordenes que mutan el estado; control de flujo explicito	C, Pascal, COBOL
Funcional	Funciones puras matematicas, inmutabilidad, sin efectos colaterales, recursion y funciones de orden superior	LISP, Haskell, Erlang
Logico	Hechos y reglas; el motor resuelve por unificacion y retroceso (clausulas de Horn)	Prolog
Heuristico	Soluciones aproximadas suficientemente buenas para problemas NP-completos/NP-hard; metaheurísticas	Lenguajes con bibliotecas geneticas/ACO (ILOG, frameworks)
Concurrente	Procesos/hilos paralelos; exclusion mutua con semaforos, monitores, bloqueos; sincronizacion y deadlock	Ada, Java (threads), Erlang
Orientado a objetos	Abstraccion, encapsulamiento, herencia y polimorfismo; objetos con estado y comportamiento	C++, Java, C#, Smalltalk

6. Programacion web clasica

Nacimiento de la web

Tim Berners-Lee (CERN, 1989-1991) crea la WWW sobre 3 pilares: **HTML** (formato de documentos hipertexto), **HTTP** (protocolo de transferencia, sin estado) y **URL** (direccionamiento universal). El navegador **Mosaic** (1993) populariza la web; Netscape (1994) y luego IE inician la primera guerra de navegadores.

- HTML**: lenguaje de marcado por etiquetas derivado de SGML; versiones HTML 2.0 (1995), HTML 3.2, HTML 4.01 (1999, con hojas de estilo CSS separando contenido y presentacion).
- JavaScript**: creado por Brendan Eich en Netscape (1995); lenguaje de script interpretado en el cliente; estandarizado como ECMAScript.
- DHTML (Dynamic HTML)**: no es un estandar sino la combinacion de **HTML + CSS + JavaScript + DOM** para modificar la pagina en el cliente sin recargarla.

PHP clasico con MySQL

```
$cn = mysql_connect($host, $user, $pass); // abre socket al servidor MySQL
mysql_select_db("arsenales", $cn);
$rs = mysql_query("SELECT * FROM personal", $cn);
// al finalizar el script el socket se CIERRA automaticamente

$fp = mysql_pconnect($host, $user, $pass); // conexion PERISTENTE:
// PHP la devuelve del pool de Apache si ya existe (no repite
// el handshake TCP ni la autentificacion al terminar el script)
```

- Diferencia critica**: `mysql_connect()` cierra la conexion al finalizar el script; `mysql_pconnect()` la mantiene en el pool del servidor web para reutilizarla en peticiones posteriores (menor overhead, riesgo de saturar `max_connections` si se abusa).
- Vigencia**: las funciones `mysql_*` quedaron obsoletas en PHP 5.5 y se **eliminaron en PHP 7**; hoy se usan **mysqli** o **PDO** con consultas preparadas (defensa contra inyeccion SQL).

ASP clasico con VBScript

Objeto intrinseco	Función
Request	Lee entrada del cliente: formularios, querystring, cookies
Response	Escribe salida (Response.Write), redirecciones y cookies
Application	Variables globales compartidas por TODAS las sesiones del proceso IIS (ej. contadores); bloqueadas con Lock/Unlock
Session	Variables privadas por usuario , atadas a la cookie de sesion, con timeout (por defecto 20 min)
Server	Utilidades: Server.CreateObject, Server.MapPath

Bibliografía oficial de referencia (Programa CMN): V. J. Eslava Muñoz, El nuevo PHP paso a paso y El nuevo PHP. Conceptos avanzados (2013) · publicaciones de referencia sobre HTML/DHTML/JavaScript y ASP-VBScript en IIS.